

Lista de Monitoria 1 — Gabarito

Análise de Algoritmos

SCC-0201 — Introdução à Ciência da Computação II

Parte 1 — Complexidade Assintótica

Questão 1. Solução:

O *método empírico* consiste em rodar o código de fato, cronometrando tempo e medindo memória gasta na prática. Ele depende da máquina, do sistema operacional, da linguagem e da carga de dados usada no teste.

O *método analítico* não executa o código: conta-se o número de operações realizadas em função do tamanho da entrada n , chegando a uma expressão matemática. Essa contagem é independente de hardware, SO ou linguagem.

Compara-se *algoritmos* e não *programas* porque o objetivo é avaliar a ideia/estratégia de solução do problema, isolando-a de fatores externos (hardware rápido, boa implementação, compilador otimizado) que poderiam mascarar um algoritmo ruim ou desvalorizar um algoritmo bom.

Questão 2. Solução:

(a)

n	1	10	100	1000	10000
$f(n) = n^2 + n$	2	110	10100	1001000	100010000
$g(n) = 1000n$	1000	10000	100000	1000000	10000000

(b) Igualando $n^2 + n = 1000n \Rightarrow n^2 = 999n \Rightarrow n = 999$. Logo, para $n \geq 999$ (aproximadamente $n = 1000$), temos $f(n) \geq g(n)$.

(c) Queremos c e n_0 tais que $1000n \leq c \cdot n^2$ para todo $n \geq n_0$.

Escolhendo $c = 1$ e $n_0 = 1000$:

$$1000n \leq n^2 \iff 1000 \leq n$$

que é verdade para todo $n \geq 1000$. Logo $1000n = O(n^2)$ com $c = 1$, $n_0 = 1000$.

Questão 3. Solução:

(a) $f(n) = 5n^3 + 2n$, $g(n) = n^3$. O termo dominante de $f(n)$ é n^3 , mesma ordem de $g(n)$. Logo:

$$f(n) = \Theta(n^3)$$

(b) $f(n) = n \log n$, $g(n) = n^2$. Como $n \log n$ cresce mais devagar que n^2 (pois $\log n = o(n)$):

$$f(n) = O(g(n)) \quad \text{mas não } \Omega(g(n))$$

(c) $f(n) = 2^n$, $g(n) = n^{10}$. Função exponencial sempre supera qualquer polinômio para n suficientemente grande:

$$f(n) = \Omega(g(n)) \quad (\text{na verdade } f(n) = \omega(g(n)), \text{ cresce estritamente mais rápido})$$

(d) $f(n) = \log n$, $g(n) = \log^2 n$. Como $\log n < \log^2 n$ para n grande (pois $\log n \rightarrow \infty$):

$$f(n) = O(g(n)) \quad \text{mas não } \Omega(g(n))$$

Questão 4. Solução:

Queremos c_1, c_2, n_0 tais que

$$c_1 n^2 \leq \frac{n^2}{3} - 2n \leq c_2 n^2, \quad \forall n \geq n_0.$$

Dividindo tudo por n^2 (válido para $n > 0$):

$$c_1 \leq \frac{1}{3} - \frac{2}{n} \leq c_2$$

Lado direito: $\frac{1}{3} - \frac{2}{n} \leq \frac{1}{3}$ sempre vale (pois $2/n > 0$). Basta escolher $c_2 = \frac{1}{3}$.

Lado esquerdo: precisamos de $c_1 \leq \frac{1}{3} - \frac{2}{n}$. Escolhendo $c_1 = \frac{1}{21}$, a desigualdade

$$\frac{1}{21} \leq \frac{1}{3} - \frac{2}{n}$$

é satisfeita para $n \geq 7$ (basta isolar n : $\frac{2}{n} \leq \frac{1}{3} - \frac{1}{21} = \frac{6}{21} \Rightarrow n \geq 7$).

Logo, com $c_1 = \frac{1}{21}$, $c_2 = \frac{1}{3}$ e $n_0 = 7$, concluímos:

$$g(n) = \Theta(n^2)$$

Questão 5. Solução:

Ordenando da menor para a maior taxa de crescimento:

$$\log n < \log^2 n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$$

Questão 6. Solução:

Dividindo cada função por n :

$$\frac{n^{1.5}}{n} = n^{0.5}, \quad \frac{n \log n}{n} = \log n$$

Elevando ao quadrado para comparar mais facilmente:

$$(n^{0.5})^2 = n, \quad (\log n)^2 = \log^2 n$$

Como n cresce mais rápido que $\log^2 n$ para qualquer n suficientemente grande (nenhuma potência de $\log$ alcança uma função linear), concluímos que:

$$n \log n = o(n^{1.5})$$

Ou seja, o algoritmo novo ($n \log n$) é assintoticamente mais eficiente e deve ser o adotado — a diferença de desempenho tende a aumentar conforme o volume de dados cresce, o que pode ser decisivo em produção.

Questão 7. Solução:

(a) Comandos sequenciais têm seu tempo somado: $T_1(n) + T_2(n)$. Como cada $T_i(n)$ é limitado por sua respectiva função ($f(n)$ e $g(n)$), o tempo total nunca ultrapassa duas vezes o maior dos dois — e constantes multiplicativas são descartadas na notação assintótica. Por exemplo:

```
for i = 1 to n:      // custa  $O(n)$ 
    ...
for i = 1 to n:
    for j = 1 to n:  // custa  $O(n^2)$ 
        ...
```

O primeiro laço custa $O(n)$, o segundo $O(n^2)$; juntos, o tempo total é dominado pelo maior: $O(n^2) = O(\max(f(n), g(n)))$.

(b) Laços aninhados multiplicam os custos porque o laço interno é executado por completo para *cada* iteração do laço externo:

```
for i = 1 to n:      //  $O(n)$ 
    for j = 1 to n:  //  $O(n)$  a cada execução do laço externo
        ...
```

Total: n execuções do laço externo $\times n$ do interno $= O(n \cdot n) = O(f(n) \cdot g(n))$.

Questão 8. Solução:

O laço externo executa n vezes. Para cada execução dele, o laço interno também executa n vezes. A operação ‘soma = soma + i*j’ custa tempo constante $O(1)$.

Total de operações:

$$\sum_{i=1}^n \sum_{j=1}^n O(1) = n \cdot n \cdot O(1) = O(n^2)$$

Logo, a complexidade do trecho é $O(n^2)$.

Questão 9. Solução:

$T_{melhor}(n)$: menor tempo possível para uma entrada de tamanho n (situação mais favorável).

$T_{mdio}(n)$: tempo esperado considerando a distribuição de todas as entradas possíveis de tamanho n .

$T_{pior}(n)$: maior tempo possível para uma entrada de tamanho n (situação mais desfavorável).

Sempre vale $T_{melhor}(n) \leq T_{mdio}(n) \leq T_{pior}(n)$.

Usa-se $T_{pior}(n)$ na prática porque ele fornece uma garantia válida para *qualquer* entrada, incluindo os casos mais problemáticos — o que é essencial em sistemas críticos ou de uso constante. Além disso, $T_{mdio}(n)$ é normalmente mais difícil de calcular, pois exige conhecer a distribuição de probabilidade das entradas, e $T_{melhor}(n)$ tem pouca utilidade prática, já que não representa uma garantia.

Parte 2 — Recorrências

Questão 10. Solução:

Uma recursão "disfarçada" de laço é aquela em que cada chamada recursiva reduz o problema por uma quantidade fixa (geralmente $n - 1$) e faz apenas uma chamada recursiva, sem combinar múltiplos subproblemas — o comportamento é idêntico a um laço simples. Exemplo: calcular o fatorial recursivamente,

$$fat(n) = n \cdot fat(n - 1)$$

tem exatamente o mesmo custo de um laço 'for' de 1 a n : $O(n)$.

Já uma recursão do tipo dividir-e-conquistar quebra o problema em *múltiplos* subproblemas menores (tipicamente frações do tamanho original, como $n/2$), resolve cada um recursivamente e depois combina as soluções. Exemplo: o Mergesort, que divide o vetor ao meio, ordena cada metade recursivamente e depois intercala os resultados.

Questão 11. Solução:

(a) Fibonacci (recursivo ingênuo):

$$T(n) = T(n - 1) + T(n - 2) + 6, \quad T(n) = O(2^n)$$

(b) Busca binária:

$$T(n) = T(n/2) + O(c), \quad T(n) = O(\log n)$$

(c) Mergesort:

$$T(n) = 2T(n/2) + n, \quad T(n) = O(n \log n)$$

Questão 12. Solução (Método da Substituição):

Palpite: $T(n) \leq cn \log n$ para alguma constante $c > 0$ e n suficientemente grande.

Base da indução: para $n = 1$, $T(1) = 1$. Como $n \log n = 0$ quando $n = 1$, tratamos a base separadamente (é comum assumir $T(n) = O(1)$ para valores pequenos de n , fora da análise assintótica).

Passo indutivo: suponha que a hipótese vale para todo valor menor que n , em particular para $n/2$:

$$T(n/2) \leq c(n/2) \log(n/2)$$

Substituindo na recorrência:

$$T(n) = 2T(n/2) + n \leq 2 \left[c \frac{n}{2} \log \frac{n}{2} \right] + n = cn \log \frac{n}{2} + n$$

Usando $\log(n/2) = \log n - 1$:

$$T(n) \leq cn(\log n - 1) + n = cn \log n - cn + n$$

Queremos que isso seja $\leq cn \log n$. Basta que:

$$-cn + n \leq 0 \iff c \geq 1$$

Logo, escolhendo qualquer $c \geq 1$, a hipótese indutiva se sustenta. Concluimos:

$$T(n) = O(n \log n)$$

Questão 13. Solução (Método da Árvore de Recursão):

A recorrência $T(n) = 2T(n/2) + n$ gera uma árvore em que:

- **Nível 0:** 1 nó, custo n (o "+n" da chamada original).
- **Nível 1:** 2 nós, cada um com subproblema de tamanho $n/2$, custo $n/2$ cada $\Rightarrow$ total $2 \cdot (n/2) = n$.
- **Nível 2:** 4 nós, cada um de tamanho $n/4$, custo $n/4$ cada $\Rightarrow$ total $4 \cdot (n/4) = n$.
- **Nível k :** 2^k nós de tamanho $n/2^k$, custo total n (o mesmo em todos os níveis).

A recursão termina quando o subproblema tem tamanho 1, ou seja, quando $n/2^k = 1 \Rightarrow k = \log n$.

Portanto, a árvore tem $\log n + 1$ níveis, e cada nível custa n . Somando tudo:

$$T(n) = \underbrace{n + n + \dots + n}_{\log n + 1 \text{ vezes}} = n \log n + n$$

Descartando o termo de menor ordem:

$$T(n) = \Theta(n \log n)$$

Questão 14. Solução (Método Mestre):

Lembrando o Método Mestre para $T(n) = aT(n/b) + f(n)$, comparando $f(n)$ com $n^{\log_b a}$.

(a) $T(n) = 4T(n/2) + n$: $a = 4$, $b = 2$, $f(n) = n$.

$$n^{\log_b a} = n^{\log_2 4} = n^2$$

Como $f(n) = n = O(n^{2-1})$ (Caso 1, com $x = 1$):

$$T(n) = \Theta(n^2)$$

(b) $T(n) = 2T(n/2) + n^2$: $a = 2$, $b = 2$, $f(n) = n^2$.

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Como $f(n) = n^2 = \Omega(n^{1+1})$ e a condição de regularidade se verifica (Caso 3):

$$T(n) = \Theta(n^2)$$

(c) $T(n) = 8T(n/2) + n^3$: $a = 8$, $b = 2$, $f(n) = n^3$.

$$n^{\log_b a} = n^{\log_2 8} = n^3$$

Como $f(n) = \Theta(n^3) = \Theta(n^{\log_b a})$ (Caso 2):

$$T(n) = \Theta(n^3 \log n)$$

(d) $T(n) = 3T(n/4) + n \log n$: $a = 3$, $b = 4$, $f(n) = n \log n$.

$$n^{\log_b a} = n^{\log_4 3} \approx n^{0.79}$$

Como $f(n) = n \log n$ cresce mais rápido que $n^{0.79}$, ou seja, $f(n) = \Omega(n^{0.79+x})$ para algum $x > 0$, e a condição de regularidade se aplica (Caso 3):

$$T(n) = \Theta(n \log n)$$

Questão 15. Solução:

O Método Mestre exige que $f(n)$ seja *polinomialmente* maior ou menor que $n^{\log_b a}$ (isto é, difira por um fator n^x para algum $x > 0$ real) para se encaixar nos Casos 1 ou 3, ou seja *exatamente igual* (a menos de constante) para o Caso 2.

Na recorrência $T(n) = 2T(n/2) + n/\log n$, temos $a = 2$, $b = 2$, então $n^{\log_b a} = n$. Comparando com $f(n) = n/\log n$:

- $f(n)$ é *menor* que n , mas apenas por um fator logarítmico ($1/\log n$), não por um fator polinomial n^{-x} .
- Logo, $f(n) \neq O(n^{1-x})$ para nenhum $x > 0$ (não se encaixa no Caso 1).
- Também não é $\Theta(n)$ (não se encaixa no Caso 2).

Existe uma "lacuna" entre os casos que o Método Mestre não cobre — situações em que a diferença é apenas um fator logarítmico. Nesses casos, é necessário usar outra técnica (como o Método da Árvore de Recursão) para resolver a recorrência.

Questão 16. Solução:

A recorrência $T(n) = T(n-1) + T(n-2) + 6$ faz **duas** chamadas recursivas a cada nível, cada uma reduzindo o tamanho do problema em apenas 1 unidade. Isso gera uma árvore de recursão que praticamente dobra de tamanho a cada nível (semelhante à árvore de chamadas do Fibonacci), resultando em aproximadamente 2^n chamadas no total — daí a complexidade $O(2^n)$.

Isso ocorre porque o mesmo subproblema (por exemplo, calcular Fibonacci de um valor k) é recalculado **repetidamente** em diferentes ramos da árvore, sem que o resultado seja reaproveitado.

Já a versão iterativa mantém apenas os dois últimos valores calculados e os atualiza em um laço simples, sem recomputação: cada valor de Fibonacci é calculado exatamente uma vez, resultando em $O(n)$.

Esse contraste mostra que a escolha do **algoritmo** (estratégia usada) é muito mais determinante para o desempenho do que a linguagem de programação ou o hardware utilizado — um algoritmo exponencial jamais será "salvo" por rodar em um computador mais rápido, para entradas suficientemente grandes.